

JAVA PROGRAMMING – COMPLETE EASY NOTES

Java is a high-level, pure object-oriented programming language.

It was developed by James Gosling in 1996.

Features of Java:

- ✓ General purpose language
 - ✓ Platform independent language
 - ✓ Object-oriented programming language
 - ✓ Secure and robust language
-

BASIC JAVA SYNTAX

Every Java program starts with a class and main method.

Syntax:

```
class ClassName
{
    public static void main(String args[])
    {
        // Code
    }
}
```

👉 main() method is the starting point of Java program execution.

COMMENTS IN JAVA

Comments are used to describe code and are ignored by compiler.

Types of Comments:

1. Single Line Comment
 2. Multiline Comment
-

SINGLE LINE COMMENT

Used for writing comments in one line.

Syntax:

```
// Comment
```

Example:

```
// This is Java program
```

MULTILINE COMMENT

Used for writing comments in multiple lines.

Syntax:

```
/*
```

Multiple line

Comment

```
*/
```

INPUT AND OUTPUT IN JAVA

Java provides different methods for taking input and displaying output.

Input classes:

✓ Scanner

✓ Console

OUTPUT STATEMENT

Used to display output on screen.

Syntax:

```
System.out.print("Output Statement");
```

Example:

```
System.out.print("Hello Java");
```

SCANNER CLASS

Scanner is used to take input from the user.

First import Scanner package:

```
import java.util.Scanner;
```

Create Scanner object:

```
Scanner objectname = new Scanner(System.in);
```

Taking input:

```
Variable = objectname.nextDatatype();
```

SCANNER INPUT METHODS

Method	Input Type
nextInt()	Integer value
nextLine()	String value
nextDouble()	Double value
nextFloat()	Float value
nextBoolean()	Boolean value

Example:

```
Scanner sc = new Scanner(System.in);
```

```
int age = sc.nextInt();
```

CONSOLE CLASS

Console class is another way to take input from user.

Import:

```
import java.io.Console;
```

Create object:

```
Console objectname = System.console();
```

Input methods:

Method	Data Type
--------	-----------

readLine()	String
------------	--------

readInt()	Integer
-----------	---------

readFloat()	Float
-------------	-------

readDouble()	Double
--------------	--------

readBoolean()	Boolean
---------------	---------

DATATYPES IN JAVA

Datatype defines the type of data stored in a variable.

Main Java datatypes:

✓ int

✓ float

✓ char

✓ double

✓ String

✓ Array

KEYWORDS IN JAVA

Keywords are reserved words having predefined meaning in Java.

Java contains mainly 50 keywords.

Examples:

class

public

static

void

int

if

else

VARIABLE DECLARATION AND INITIALIZATION

A variable is a memory location used to store data.

Syntax:

```
Datatype variableName = value;
```

Example:

```
int age = 20;
```

ARRAY DECLARATION AND INITIALIZATION

Array stores multiple values of the same datatype.

Syntax:

```
Datatype arrayName[] = new Datatype[size];
```

Example:

```
int marks[] = new int[5];
```

STRING DECLARATION

String is used to store a group of characters.

Two methods:

1. Using new keyword

Syntax:

```
String name = new String("Value");
```

2. Direct assignment

Syntax:

```
String name = "Value";
```

Example:

```
String city = "Pune";
```

FUNCTION / METHOD IN JAVA

A function is a block of code used to perform a specific task.

Syntax:

```
ReturnType functionName(arguments)
```

```
{  
    // Code block  
}
```

Example:

```
void display()  
{  
    System.out.println("Hello");  
}
```

FUNCTION CALLING

Calling a function means executing the function.

Syntax:

```
functionName(arguments);
```

Example:

```
display();
```

CONDITIONAL STATEMENTS IN JAVA

Conditional statements are used to make decisions based on conditions.

Types:

1. IF Statement
 2. IF ELSE Statement
 3. IF ELSE IF Statement
 4. Nested IF Statement
-

IF STATEMENT

Used when a block executes only when condition is true.

Syntax:

```
if(condition)
{
    Statements;
}
```

Example:

```
if(age>=18)
{
    System.out.println("Eligible");
}
```

IF ELSE STATEMENT

Used when there are two conditions.

Syntax:

```
if(condition)
{
    Statements;
}
else
{
    Statements;
}
```

IF ELSE IF STATEMENT

Used for multiple condition checking.

Syntax:

```
if(condition1)
```

```
{  
    Statements;  
}  
else if(condition2)  
{  
    Statements;  
}  
else  
{  
    Statements;  
}
```

NESTED IF STATEMENT

An if statement inside another if statement is called nested if.

Syntax:

```
if(condition1)  
{  
    if(condition2)  
    {  
        Statements;  
    }  
    else  
    {  
        Statements;  
    }  
}  
else  
{
```



```
Statements;  
}
```

☆ EASY WAY TO REMEMBER

IF → One condition

IF ELSE → Two choices

IF ELSE IF → Multiple choices

Nested IF → IF inside IF

LOOPING STATEMENTS IN JAVA

Loops are used to execute a block of code repeatedly until a condition becomes false.

Types of loops:

- ✓ while loop
 - ✓ do while loop
 - ✓ for loop
 - ✓ for each loop
-

WHILE LOOP

While loop executes statements while the condition is true.

Syntax:

```
while(condition)  
{  
    Statements;  
}
```

Example:

```
while(i<=5)  
{  
    System.out.println(i);
```

```
    i++;  
}
```

DO WHILE LOOP

Do while loop executes the block at least one time before checking the condition.

Syntax:

```
do  
{  
    Statements;  
}  
while(condition);
```

Example:

```
do  
{  
    System.out.println(i);  
    i++;  
}  
while(i<=5);
```

FOR LOOP

For loop is used when the number of iterations is known.

Syntax:

```
for(initialization; condition; increment/decrement)  
{  
    Statements;  
}
```

Example:

```
for(int i=1;i<=5;i++)
```

```
{  
    System.out.println(i);  
}
```

FOR EACH LOOP

For each loop is used to print array elements easily.

Syntax:

```
for(Datatype variable : arrayName)  
{  
    Statements;  
}
```

Example:

```
for(int value : marks)  
{  
    System.out.println(value);  
}
```

☆ EASY WAY TO REMEMBER LOOPS

while → Condition based loop

do while → Executes minimum one time

for → Fixed number of repetitions

for each → Array elements printing

SWITCH CASE STATEMENT

Switch statement is used to select one option from multiple choices.

Syntax:

```
switch(expression)  
{
```

```
case value1:
    Statements;
    break;
```

```
case value2:
    Statements;
    break;
```

```
default:
    Statements;
    break;
```

```
}
```

Example:

```
switch(day)
```

```
{
```

```
case 1:
    System.out.println("Monday");
    break;
```

```
default:
    System.out.println("Invalid");
```

```
}
```

OPERATORS IN JAVA

Operators are symbols used to perform operations on variables and values.

Types of operators:

1. Arithmetic Operators
2. Relational Operators

3. Logical Operators
 4. Increment/Decrement Operators
 5. Ternary Operators
 6. Assignment Operators
-

ARITHMETIC OPERATORS

Used for mathematical operations.

Operator Meaning

+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus

Example:

```
int c = a + b;
```

RELATIONAL OPERATORS

Used to compare two values.

Operator Meaning

>	Greater than
<	Less than
>=	Greater than equal
<=	Less than equal
==	Equal
!=	Not Equal

Example:

```
if(a>b)
```

```
{
```

```
}
```

LOGICAL OPERATORS

Used to combine multiple conditions.

Operator Meaning

&& Logical AND

! Logical NOT

Example:

```
if(age>18 && age<60)
```

```
{
```

```
}
```

INCREMENT / DECREMENT OPERATORS

Used to increase or decrease value by one.

Operators:

++ Increment

-- Decrement

Example:

```
i++;
```

TERNARY OPERATOR

Ternary operator is a short form of if else.

Operators:

?

:

Syntax:

condition ? true_value : false_value;

Example:

result = (a>b)?a:b;

ASSIGNMENT OPERATOR

Used to assign values to variables.

Operator:

=

Example:

int a = 10;

OBJECT ORIENTED PROGRAMMING (OOP)

Java is a pure object-oriented programming language.

OOP concepts:

- ✓ Class and Object
 - ✓ Encapsulation
 - ✓ Inheritance
 - ✓ Abstraction
 - ✓ Polymorphism
-

CLASS AND OBJECT

CLASS

Class is a blueprint used to create objects.

Syntax:

class ClassName

{

 DataMembers;

```
    Methods;  
}
```

Example:

```
class Student
```

```
{  
    int id;  
  
    void display()  
    {  
    }  
}
```

OBJECT

Object is an instance of a class.

Syntax:

```
ClassName objectName = new ClassName();
```

Example:

```
Student s = new Student();
```

ACCESSING OBJECT MEMBERS

Calling method:

```
objectName.methodName();
```

Accessing data member:

```
objectName.dataMemberName;
```

☆ EASY WAY TO REMEMBER

Class → Blueprint

Object → Real thing created from class

Example:

Class = House Design

Object = Actual House

ENCAPSULATION

Encapsulation means binding multiple data members and methods into a single unit.

Class contains:

✓ Variables

✓ Methods

Syntax:

```
class ClassName
```

```
{
```

```
    DataMembers;
```

```
    MemberFunction()
```

```
{
```

```
}
```

```
}
```

INHERITANCE

Inheritance means creating a new class from an existing class.

It improves code reusability.

Keyword used:

extends

Syntax:

```
class ParentClass
```

```
{  
    Properties;  
}
```

class ChildClass extends ParentClass

```
{  
    Properties;  
}
```

TYPES OF INHERITANCE IN JAVA

Java supports:

- ✓ Single Inheritance
- ✓ Multilevel Inheritance
- ✓ Hierarchical Inheritance

Java does not support directly:

- ✗ Multiple Inheritance
- ✗ Hybrid Inheritance

Multiple inheritance can be achieved using Interface.

☆ EASY WAY TO REMEMBER

Inheritance = Parent properties → Child class

extends keyword = Creates relationship

DATA ABSTRACTION IN JAVA

Abstraction means hiding low-level implementation details and showing only important information to users.

Access sections:

- ✓ Public
- ✓ Private
- ✓ Protected

Syntax:

```
class ClassName  
  
{  
    accessSpecifier datatype variableName;  
}
```

POLYMORPHISM IN JAVA (VERY IMPORTANT)

Polymorphism means showing different behavior according to different situations.

Types of Polymorphism:

1. Compile Time Polymorphism
2. Runtime Polymorphism

👉 In polymorphism, inheritance is generally present.

METHOD OVERLOADING (FUNCTION OVERLOADING)

Method overloading means:

Same class contains multiple methods with the same name but different parameters.

It is a compile-time polymorphism.

Syntax:

```
class ClassName  
  
{  
    void methodName()  
    {  
        // No argument  
    }  
}
```

```
void methodName(int value)
{
    // With argument
}
}
```

Example:

```
void add()
{
}
```

```
void add(int a)
{
}
```

SUPER KEYWORD IN JAVA

The super keyword is used to access parent class members.

Uses of super keyword:

- ✓ Access parent class variable
- ✓ Access parent class method
- ✓ Call parent class constructor

Syntax:

```
super.variableName;
```

```
super.methodName();
```

```
super();
```

SUPER METHOD

Super method is used to access the constructor of the parent class using a child class object.

Example:

```
super();
```

METHOD OVERRIDING

Method overriding means:

Multiple classes contain the same method name with the same parameters.

It is runtime polymorphism.

Inheritance is required.

Syntax:

```
class Parent
```

```
{  
    void display()  
    {  
    }  
}
```

```
class Child extends Parent
```

```
{  
    void display()  
    {  
    }  
}
```

CONSTRUCTOR IN JAVA

Constructor is a special type of function.

Features:

- ✓ Name is same as class name
- ✓ Executes automatically during object creation
- ✓ Has no return type

Types:

1. Default Constructor
2. Parameterized Constructor

DEFAULT CONSTRUCTOR

A constructor without arguments is called a default constructor.

Syntax:

```
ClassName objectName =  
new ClassName();
```

Example:

```
Student s = new Student();
```

PARAMETERIZED CONSTRUCTOR

A constructor containing arguments is called parameterized constructor.

Syntax:

```
ClassName objectName =  
new ClassName(datatype variable);
```

Example:

```
Student s = new Student(10);
```

CONSTRUCTOR OVERLOADING

Same class contains multiple constructors with different parameters.

Syntax:

```
class ClassName  
{
```

```
ClassName()  
{  
    // Default constructor  
}  
  
ClassName(int value)  
{  
    // Parameterized constructor  
}  
}
```

THIS KEYWORD

this keyword is used to refer to the current object.

Main use:

✓ Differentiate local variable and instance variable.

Syntax:

this.instanceVariable = localVariable;

Example:

class Student

```
{  
    int id;  
  
    Student(int id)  
    {  
        this.id=id;  
    }  
}
```

THIS METHOD

this() method is used to call one constructor from another constructor.

Rules:

✓ Written as first statement inside constructor.

Syntax:

```
class ClassName
```

```
{
```

```
    ClassName()
```

```
    {
```

```
    }
```

```
    ClassName(int value)
```

```
    {
```

```
        this();
```

```
    }
```

```
}
```

FINALIZE METHOD

Finalize method is used to deallocate memory occupied by objects.

Syntax:

```
objectName = null;
```

It helps garbage collector to remove unused objects.

FINAL KEYWORD IN JAVA

final keyword is used to restrict changes.

Types:

1. Final Variable

2. Final Method

3. Final Class

FINAL VARIABLE

Value of final variable cannot be changed.

Syntax:

```
final datatype variableName = value;
```

Example:

```
final int age = 20;
```

FINAL METHOD

Final method cannot be overridden by child class.

Syntax:

```
final returnType methodName()  
{  
  
}
```

FINAL CLASS

Final class cannot be inherited.

Syntax:

```
final class ClassName  
{  
  
}
```

EXCEPTION HANDLING IN JAVA

Exception handling is used to handle runtime errors and prevent abnormal program termination.

Keywords:

- ✓ try
- ✓ catch
- ✓ throw
- ✓ finally
- ✓ throws

TRY CATCH BLOCK

Syntax:

```
try
{
    Exception code;
}
```

```
catch(Exception e)
{
    Handling code;
}
```

USER DEFINED PACKAGE IN JAVA

Package is used to group related classes and interfaces.

PACKAGE CREATION

Syntax:

```
package packagename.projectname;
```

Example:

```
package project.musicplayer;
```

COMPILING PACKAGE

Syntax:

```
javac -d . ProgramName.java
```

ACCESSING PACKAGE

Steps:

1. Import package

```
import packageName;
```

2. Create object

```
ClassName obj =
```

```
new ClassName();
```

3. Call methods

```
obj.methodName();
```

RULES OF PACKAGE

- ✓ Package statement must be first statement.
 - ✓ Class and methods inside package should be public.
 - ✓ Package should not contain main method.
-

☆ EASY WAY TO REMEMBER

Package = Collection of related classes

import = Use package

object = Access package class

ABSTRACTION IN JAVA (ADVANCED)

Abstraction is the process of hiding implementation details and showing only essential information.

Java provides abstraction using:

- ✓ Abstract Class
 - ✓ Abstract Method
 - ✓ Interface
-

CONCRETE CLASS

A concrete class is a normal class that contains:

- ✓ Declaration of methods
- ✓ Definition of methods

Example:

```
class Student
{
    void display()
    {
        System.out.println("Hello");
    }
}
```

ABSTRACT METHOD

An abstract method is a method that is declared in parent class and defined in child class.

Features:

- ✓ It has only declaration
- ✓ It does not contain method body
- ✓ Uses abstract keyword

Syntax:

```
abstract void methodName();
```

Example:

```
abstract void display();
```

ABSTRACT CLASS

An abstract class is a class that contains at least one abstract method.

Features:

- ✓ Cannot create object directly
- ✓ Can contain abstract and normal methods

Syntax:

```
abstract class ClassName  
{  
  
  
}
```

Example:

```
abstract class Animal  
{  
    abstract void sound();  
}
```

PURE ABSTRACT CLASS

A pure abstract class contains only abstract methods.

Example:

```
abstract class Shape  
{  
    abstract void draw();  
}
```

INTERFACE IN JAVA

Interface is like a class that contains method declarations.

It is used to achieve abstraction and multiple inheritance.

Syntax:

```
interface InterfaceName
```

```
{
```

```
    Method declaration;
```

```
}
```

Example:

```
interface Vehicle
```

```
{
```

```
    void run();
```

```
}
```

IMPLEMENTATION OF INTERFACE

The implements keyword is used to implement an interface.

Syntax:

```
class ClassName implements InterfaceName
```

```
{
```

```
}
```

Example:

```
class Bike implements Vehicle
```

```
{
```

```
    public void run()
```

```
    {
```

```
    }
```

```
}
```

FEATURES OF INTERFACE

Methods inside interface:

- ✓ Public
- ✓ Abstract

Variables inside interface:

- ✓ Public
- ✓ Static
- ✓ Final

☆ EASY WAY TO REMEMBER

Abstract Class → Partial abstraction

Interface → Complete abstraction

implements → Used with interface

extends → Used with class inheritance

JAVA COMMAND LINE EXECUTION

Java program execution has two steps:

1. Compilation
2. Execution

COMPILATION

Used to convert Java source code into bytecode.

Command:

```
javac ProgramName.java
```

EXECUTION

Used to run bytecode.

Command:

```
java ProgramName
```

JAVA DATABASE CONNECTIVITY (JDBC)

JDBC is used to connect Java applications with databases.

JDBC provides permanent data storage.

JDBC works as an intermediate between:

Frontend ↔ Backend Database

COMPONENTS OF JDBC

There are mainly 4 components:

1. JDBC API
 2. JDBC Driver
 3. JDBC Driver Manager
 4. JDBC Test Suite
-

JDBC API

JDBC API provides classes and interfaces for database connectivity.

Packages:

`java.sql.*;`

`javax.sql.*;`

JDBC DRIVER

JDBC Driver converts Java program instructions into database-understandable format.

It creates communication between:

Java Application ↔ Database

JDBC DRIVER MANAGER

Driver Manager manages JDBC drivers according to the database.

JDBC TEST SUITE

It loads JDBC drivers according to database requirements.

JDBC ARCHITECTURE

Flow of JDBC:

Java Application



JDBC API



JDBC Driver Manager



Database

TYPES OF JDBC DRIVERS

There are mainly 4 types:

1. Type 1 Driver
 2. Type 2 Driver
 3. Type 3 Driver
 4. Type 4 Driver
-

TYPE 4 JDBC DRIVER

Type 4 driver is the best JDBC driver.

Reason:

- ✓ Directly communicates with database
 - ✓ Faster performance
 - ✓ Platform independent
-

STRUCTURE OF JDBC PROGRAM

Steps:

1. Import JDBC package
 2. Load JDBC driver
 3. Establish connection
 4. Create statement
 5. Execute query
 6. Process result
 7. Close connection
-

STEP 1: IMPORT JDBC PACKAGE

Syntax:

```
import java.sql.*;
```

STEP 2: LOAD JDBC DRIVER

Syntax:

```
Class.forName("DriverName");
```

Example:

```
Class.forName("oracle.jdbc.driver.OracleDriver");
```

STEP 3: ESTABLISH CONNECTION

Connection interface is used.

Syntax:

```
Connection con =  
DriverManager.getConnection(  
databaseURL,  
username,  
password  
);
```

STEP 4: CREATE STATEMENT

Statement interface is used to prepare SQL queries.

Syntax:

```
Statement state =  
con.createStatement();
```

STEP 5: EXECUTE QUERY

Methods:

executeUpdate()

Used for:

✓ INSERT

✓ UPDATE

✓ DELETE

Syntax:

```
state.executeUpdate("query");
```

executeQuery()

Used for SELECT operation.

Syntax:

```
state.executeQuery("query");
```

execute()

Used for all types of operations.

Syntax:

```
state.execute("query");
```

STEP 6: PROCESS RESULT

Result returned from database is processed using ResultSet.

STEP 7: CLOSE CONNECTION

Syntax:

```
con.close();
```

IMPORTANT JDBC NOTE

Before creating JDBC programs:

- ✓ Edit environment variables
 - ✓ Add JDBC driver path
-

☆ FINAL SUMMARY

- ✓ Java = High-level object-oriented language
- ✓ Class = Blueprint of object
- ✓ Object = Instance of class
- ✓ Encapsulation = Binding data and methods
- ✓ Inheritance = Reusing existing class properties
- ✓ Abstraction = Hiding implementation details
- ✓ Polymorphism = Different behavior
- ✓ Constructor = Executes during object creation
- ✓ Exception Handling = Handles runtime errors

- ✓ Package = Collection of classes
- ✓ Interface = Supports abstraction and multiple inheritance
- ✓ JDBC = Connects Java with Database
- ✓ Type 4 Driver = Best JDBC driver
- ✓ Java is platform independent because of JVM

 **JAVA PROGRAMMING NOTES COMPLETED**